Work Item: MCP Custody Server v1.4.4 Raw JSON-RPC Rebuild

ID: DS-20260817-MCP-v1.4.4-JSONRPC-Rebuild
Date: 2026-08-17
Operator: Ken Tombs
Design Lead: Claude
Status: PRE-BUILD (awaiting approval)

DIAGNOSIS (Completed)

Issue Identified

FastMCP v1.29.0 tool serialization bug: Server code defines and announces 7 MCP tools, but only 6 reach Claude Code's tool roster. Root cause: `emit_documentary_gap` is truncated during MCP protocol serialization (JSON-RPC handshake).
Evidence:

- Server console: "MCP Custody Server v1.4.3 initialized and ready" lists all 7 tools
- Claude Code harness: deferred-tool inventory shows only 6 tools from dossier-custody server
- Enum schema removal did not fix the issue (confirms FastMCP serialization layer problem)
- Ledger infrastructure confirmed working (103KB of governance entries written successfully)

Impact

- Documentary gap logging blocked at MCP protocol layer
- Workaround in place: findings-file ROMER format (viable but not ideal)
- Pilot infrastructure robust except for this single serialization bug

---

DECISION

Approach: Strip FastMCP, implement raw JSON-RPC MCP protocol

Rationale:

1. FastMCP is a convenience wrapper that abstracts the MCP protocol
2. The bug is in that abstraction layer (tool serialization truncation)
3. Raw JSON-RPC gives full transparency and control
4. MCP protocol (JSON-RPC 2.0 over stdio) is simple enough to implement directly
5. Removes external dependency that's causing opacity

Alternative considered and rejected:

- Further FastMCP debugging: diminishing returns, time cost high, FastMCP is unmaintained
- Findings-file workaround: functional but not a proper fix; pilot needs robust infrastructure

---

SCOPE OF WORK

v1.4.4 Implementation

Remove:

- All `from fastmcp import` statements
- `Tool()` class definitions
- `@server.call_tool()` decorator
- `@server.list_tools()` decorator
- FastMCP initialization code

Implement:

- Raw JSON-RPC 2.0 message handler over stdio
- Tool registry as plain Python dict with JSON schema definitions
- Request router: parse `{"jsonrpc": "2.0", "method": "tools/list", ...}` → execute → return JSON response
- MCP protocol sequence:
  1. Initialize (await `initialize` request)
  2. Announce tools (via `tools/list` response)
  3. Handle tool calls (via `tools/call` requests)
  4. Stream results back as `{"result": {...}}` or error

Tool definitions (7 total):

1. `search_messages` (custodian, keyword, date_contains)
2. `list_folder` (custodian)
3. `read_file` (filepath)
4. `get_headers` (filepath)
5. `hash_document` (filepath)
6. `verify_hash` (filepath, expected_hash)
7. `emit_documentary_gap` (gap*identified, reason*identified, significance, status) ← currently truncated

Logging:

- Log every incoming JSON-RPC request (method, params)
- Log every outgoing response (success, result, error)
- Maintain existing governance ledger emissions

- File: `v1.4.4-jsonrpc-audit.log` for protocol transparency

Version metadata:

- Version string: "Dossier Space MCP Custody Server v1.4.4 (raw JSON-RPC, FastMCP removed)"
- Session ID: DS-202608xx-S12 (continuation)
- Previous: v1.4.3 (FastMCP, tool truncation bug)

SUCCESS CRITERIA

Tier 1 (Must have)

- [ ] Server starts without FastMCP imports (clean startup)
- [ ] Claude Code receives all 7 tools in deferred-tool inventory
- [ ] Tool schemas match specification exactly (no truncation)

Tier 2 (Should have)

- [ ] `emit_documentary_gap` tool call succeeds end-to-end
- [ ] Ledger entry written with `entry_type: "documentary_gap"`
- [ ] JSON-RPC audit log shows all message exchanges

Tier 3 (Nice to have)

- [ ] All 7 tools tested with real corpus data
- [ ] Performance benchmarked (startup time, message latency)
- [ ] Code documented for future maintenance

Failure criteria

- Server crashes on startup
- Claude Code receives <7 tools
- Ledger entry missing or malformed
- JSON-RPC protocol violation (invalid response format)

ARCHITECTURE

```
v1.4.4 Custody Server
  │
  ├── Ledger (write-capable, shared with governance ledger)
  ├── Corpus Reader (read-only, E:\Enron maildir)
  │
  ├── JSON-RPC Protocol Handler
  │   ├── initialize() → respond with server capabilities
  │   ├── tools/list() → return array of 7 Tool objects (JSON schema)
  │   └── tools/call(name, args) → route to method, return result
  │
  ├── Tool Methods (7)
  │   ├── search_messages()
  │   ├── list_folder()
  │   ├── read_file()
  │   ├── get_headers()
  │   ├── hash_document()
  │   ├── verify_hash()
  │   └── emit_documentary_gap() ← currently broken
  │
  └── Logging
      ├── stdout (JSON-RPC protocol trace)
      ├── session-ledger.jsonl (governance)
      └── v1.4.4-jsonrpc-audit.log (request/response audit)
```

BUILD CHECKLIST

- [ ] Create v1.4.4 from v1.4.3 template
- [ ] Remove all FastMCP imports and decorators
- [ ] Implement JSON-RPC protocol handler (initialize, tools/list, tools/call)
- [ ] Define 7 tools as JSON schema objects
- [ ] Implement each tool method (copy from v1.4.3, no changes)
- [ ] Add JSON-RPC request/response logging
- [ ] Update version string to v1.4.4
- [ ] Test: startup without errors
- [ ] Test: Claude Code receives all 7 tools
- [ ] Test: emit*documentary*gap call succeeds
- [ ] Test: ledger entry created with correct structure
- [ ] Verify governance ledger append (no corruption)

- [ ] Copy final v1.4.4 to outputs
- [ ] Update claude.json to point to v1.4.4
- [ ] Sign off completion

TIMELINE & EFFORT

Estimated:

- Design: 15 min (this document)
- Code: 45 min (raw JSON-RPC implementation)
- Testing: 20 min (startup, tool list, end-to-end call)
- **Total: ~80 minutes**

Risk: Low. JSON-RPC is simpler than FastMCP; we're removing a broken abstraction, not adding complexity.

GOVERNANCE NOTES

Why this matters:

- Pilot is supposed to be robust and auditable (Watch is never the Watched)
- Broken MCP tool means governance ledger is incomplete (documentary gaps not logged)
- Raw JSON-RPC gives full transparency into what Claude Code receives
- Every message in/out will be logged for audit

Not a scope creep:

- This is fixing an existing bug, not adding features
- The tool (emit*documentary*gap) was already designed and specified
- We're just fixing the delivery mechanism

Recorded posture (ADR-004):

- Work direct with raw JSON-RPC protocol
- Log every message exchange
- Flag divergence from MCP spec if found

SIGN-OFF

Approval: Awaiting Ken's confirmation before proceeding.

Next step: Ken confirms this plan → Claude builds v1.4.4 → test cycle → update claude.json → verify in Claude Code.

Document created: 2026-08-17 14:47 UTC
Status: Ready for build approval